

Лабораторная работа №4

Односвязный и двусвязный списки

Цель работы:

1. Реализовать шаблонный класс односвязного списка (SinglyLinkedList) с базовыми операциями
2. Реализовать шаблонный класс двусвязного списка (DoublyLinkedList) с базовыми операциями
3. Освоить операции вставки, удаления, поиска и обхода элементов в списках
4. Сравнить производительность и особенности использования односвязного и двусвязного списков
5. Научиться правильно управлять памятью при работе со списками (удаление узлов)

Теоретическое обоснование

1. Понятие связного списка

Связный список - это структура данных, в которой элементы (узлы) не хранятся в памяти непрерывно. Каждый узел содержит данные и указатель(и) на следующий (и/или предыдущий) узел.

Визуализация:

Односвязный список:

head —► [10] —► [20] —► [30] —► nullptr

Двусвязный список:

head —► [10] ⇄ [20] ⇄ [30] ⇄ nullptr
◄←—————► ◄←—————►

2. Сравнение массивов и списков

Операция	Динамический массив (vector)	Односвязный список	Двусвязный список
Доступ по индексу	O(1)	O(n)	O(n)
Вставка в начало	O(n)	O(1)	O(1)

Операция	Динамический массив (vector)	Односвязный список	Двусвязный список
Вставка в конец	$O(1)$ амортиз.	$O(1)$ с tail / $O(n)$	$O(1)$ с tail
Вставка в середину	$O(n)$	$O(n)$ поиск + $O(1)$	$O(n)$ поиск + $O(1)$
Удаление из начала	$O(n)$	$O(1)$	$O(1)$
Удаление из конца	$O(1)$	$O(n)$	$O(1)$
Поиск по значению	$O(n)$	$O(n)$	$O(n)$
Память на элемент	Только данные	Данные + указатель	Данные + 2 указателя

3. Узел односвязного списка

```
template <typename T>
struct Node {
    T data;
    Node<T>* next;

    Node(const T& value) : data(value), next(nullptr) {}
};
```

4. Узел двусвязного списка

```
template <typename T>
struct DNode {
    T data;
    DNode<T>* prev;
    DNode<T>* next;

    DNode(const T& value) : data(value), prev(nullptr), next(nullptr) {}
};
```

5. Базовые операции с односвязным списком

```
template <typename T>
class SinglyLinkedList {
private:
    Node<T>* head;
    Node<T>* tail;
    int count;

public:
    // Конструкторы и деструктор
    SinglyLinkedList() : head(nullptr), tail(nullptr), count(0) {}

    ~SinglyLinkedList() {
        clear();
    }

    // Вставка в начало O(1)
    void push_front(const T& value) {
        Node<T>* newNode = new Node<T>(value);
        newNode->next = head;
        head = newNode;
        if (tail == nullptr) tail = newNode;
        count++;
    }

    // Вставка в конец O(1) с tail
    void push_back(const T& value) {
        Node<T>* newNode = new Node<T>(value);
        if (head == nullptr) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
        count++;
    }

    // Удаление из начала O(1)
    void pop_front() {
        if (head == nullptr) throw std::underflow_error("List is empty");
        Node<T>* temp = head;
```

```

    head = head->next;
    delete temp;
    count--;
    if (head == nullptr) tail = nullptr;
}

// Удаление из конца O(n)
void pop_back() {
    if (head == nullptr) throw std::underflow_error("List is empty");
    if (head->next == nullptr) {
        delete head;
        head = tail = nullptr;
    } else {
        Node<T>* current = head;
        while (current->next != tail) current = current->next;
        delete tail;
        tail = current;
        tail->next = nullptr;
    }
    count--;
}

// Поиск O(n)
Node<T>* find(const T& value) const {
    Node<T>* current = head;
    while (current != nullptr) {
        if (current->data == value) return current;
        current = current->next;
    }
    return nullptr;
}

// Удаление по значению O(n)
bool remove(const T& value) {
    if (head == nullptr) return false;
    if (head->data == value) {
        pop_front();
        return true;
    }
    Node<T>* current = head;
    while (current->next != nullptr && current->next->data != value) {
        current = current->next;
    }
}

```

```

    }
    if (current->next == nullptr) return false;
    Node<T>* toDelete = current->next;
    current->next = toDelete->next;
    if (toDelete == tail) tail = current;
    delete toDelete;
    count--;
    return true;
}

// Вывод списка O(n)
void print() const {
    Node<T>* current = head;
    std::cout << "[";
    while (current != nullptr) {
        std::cout << current->data;
        if (current->next != nullptr) std::cout << ", ";
        current = current->next;
    }
    std::cout << "]" << std::endl;
}

void clear() {
    while (head != nullptr) {
        Node<T>* temp = head;
        head = head->next;
        delete temp;
    }
    tail = nullptr;
    count = 0;
}

int size() const { return count; }
bool empty() const { return head == nullptr; }
};

```

6. Базовые операции с двусвязным списком

```

template <typename T>
class DoublyLinkedList {
private:
    DNode<T>* head;
    DNode<T>* tail;

```

```
int count;
```

```
public:
```

```
DoublyLinkedList() : head(nullptr), tail(nullptr), count(0) {}
```

```
~DoublyLinkedList() {  
    clear();  
}
```

```
// Вставка в начало O(1)
```

```
void push_front(const T& value) {  
    DNode<T>* newNode = new DNode<T>(value);  
    if (head == nullptr) {  
        head = tail = newNode;  
    } else {  
        newNode->next = head;  
        head->prev = newNode;  
        head = newNode;  
    }  
    count++;  
}
```

```
// Вставка в конец O(1)
```

```
void push_back(const T& value) {  
    DNode<T>* newNode = new DNode<T>(value);  
    if (tail == nullptr) {  
        head = tail = newNode;  
    } else {  
        tail->next = newNode;  
        newNode->prev = tail;  
        tail = newNode;  
    }  
    count++;  
}
```

```
// Удаление из начала O(1)
```

```
void pop_front() {  
    if (head == nullptr) throw std::underflow_error("List is empty");  
    DNode<T>* temp = head;  
    head = head->next;  
    if (head != nullptr) head->prev = nullptr;  
    else tail = nullptr;  
}
```

```

        delete temp;
        count--;
    }

// Удаление из конца O(1)
void pop_back() {
    if (tail == nullptr) throw std::underflow_error("List is empty");
    DNode<T>* temp = tail;
    tail = tail->prev;
    if (tail != nullptr) tail->next = nullptr;
    else head = nullptr;
    delete temp;
    count--;
}

// Поиск O(n)
DNode<T>* find(const T& value) const {
    DNode<T>* current = head;
    while (current != nullptr) {
        if (current->data == value) return current;
        current = current->next;
    }
    return nullptr;
}

// Вставка после указанного узла O(1)
void insert_after(DNode<T>* node, const T& value) {
    if (node == nullptr) {
        push_front(value);
        return;
    }
    DNode<T>* newNode = new DNode<T>(value);
    newNode->next = node->next;
    newNode->prev = node;
    if (node->next != nullptr) node->next->prev = newNode;
    else tail = newNode;
    node->next = newNode;
    count++;
}

// Удаление указанного узла O(1)
void erase(DNode<T>* node) {

```

```

    if (node == nullptr) return;
    if (node == head) {
        pop_front();
        return;
    }
    if (node == tail) {
        pop_back();
        return;
    }
    node->prev->next = node->next;
    node->next->prev = node->prev;
    delete node;
    count--;
}

// Обход в обратном порядке
void printReverse() const {
    DNode<T>* current = tail;
    std::cout << "[";
    while (current != nullptr) {
        std::cout << current->data;
        if (current->prev != nullptr) std::cout << ", ";
        current = current->prev;
    }
    std::cout << "]" << std::endl;
}

void print() const {
    DNode<T>* current = head;
    std::cout << "[";
    while (current != nullptr) {
        std::cout << current->data;
        if (current->next != nullptr) std::cout << ", ";
        current = current->next;
    }
    std::cout << "]" << std::endl;
}

void clear() {
    while (head != nullptr) {
        DNode<T>* temp = head;
        head = head->next;
    }
}

```

```

        delete temp;
    }
    tail = nullptr;
    count = 0;
}

int size() const { return count; }
bool empty() const { return head == nullptr; }
};

```

Примеры выполнения практической части

Пример 1. Односвязный список (базовый уровень)

Задача: Реализовать класс односвязного списка с операциями push_front, push_back, pop_front, print.

```

#include <iostream>
#include <stdexcept>

template <typename T>
class SinglyLinkedList {
private:
    struct Node {
        T data;
        Node* next;
        Node(const T& value) : data(value), next(nullptr) {}
    };

    Node* head;
    Node* tail;
    int count;

public:
    SinglyLinkedList() : head(nullptr), tail(nullptr), count(0) {}

    ~SinglyLinkedList() { clear(); }

    void push_front(const T& value) {
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
        if (tail == nullptr) tail = newNode;
        count++;
    }

```

```
}
```

```
void push_back(const T& value) {  
    Node* newNode = new Node(value);  
    if (head == nullptr) {  
        head = tail = newNode;  
    } else {  
        tail->next = newNode;  
        tail = newNode;  
    }  
    count++;  
}
```

```
void pop_front() {  
    if (head == nullptr) throw std::underflow_error("List is empty");  
    Node* temp = head;  
    head = head->next;  
    delete temp;  
    count--;  
    if (head == nullptr) tail = nullptr;  
}
```

```
void print() const {  
    Node* current = head;  
    std::cout << "[";  
    while (current != nullptr) {  
        std::cout << current->data;  
        if (current->next != nullptr) std::cout << ", ";  
        current = current->next;  
    }  
    std::cout << "]" << std::endl;  
}
```

```
int size() const { return count; }  
bool empty() const { return head == nullptr; }
```

```
void clear() {  
    while (head != nullptr) {  
        Node* temp = head;  
        head = head->next;  
        delete temp;  
    }  
}
```

```

        tail = nullptr;
        count = 0;
    }
};

int main() {
    setlocale(LC_ALL, "ru");

    SinglyLinkedList<int> list;

    std::cout << "=== Односвязный список ===" << std::endl;

    list.push_back(10);
    list.push_back(20);
    list.push_back(30);
    list.push_front(5);

    std::cout << "Список: ";
    list.print();
    std::cout << "Размер: " << list.size() << std::endl;

    list.pop_front();
    std::cout << "После pop_front: ";
    list.print();

    return 0;
}

```

Ожидаемый вывод:

=== Односвязный список ===

Список: [5, 10, 20, 30]

Размер: 4

После pop_front: [10, 20, 30]

Пример 2. Двусвязный список (средний уровень)

Задача: Реализовать класс двусвязного списка с операциями push_back, pop_back, printReverse.

```

#include <iostream>
#include <stdexcept>

template <typename T>
class DoublyLinkedList {

```

private:

```
struct Node {  
    T data;  
    Node* prev;  
    Node* next;  
    Node(const T& value) : data(value), prev(nullptr), next(nullptr) {}  
};
```

```
Node* head;
```

```
Node* tail
```

```
int count;
```

public:

```
DoublyLinkedList() : head(nullptr), tail(nullptr), count(0) {}
```

```
~DoublyLinkedList() { clear(); }
```

```
void push_back(const T& value) {  
    Node* newNode = new Node(value);  
    if (tail == nullptr) {  
        head = tail = newNode;  
    } else {  
        tail->next = newNode;  
        newNode->prev = tail;  
        tail = newNode;  
    }  
    count++;  
}
```

```
void pop_back() {  
    if (tail == nullptr) throw std::underflow_error("List is empty");  
    Node* temp = tail;  
    tail = tail->prev;  
    if (tail != nullptr) tail->next = nullptr;  
    else head = nullptr;  
    delete temp;  
    count--;  
}
```

```
void print() const {  
    Node* current = head;  
    std::cout << "[";
```

```

while (current != nullptr) {
    std::cout << current->data;
    if (current->next != nullptr) std::cout << ", ";
    current = current->next;
}
std::cout << "]" << std::endl;
}

```

```

void printReverse() const {
    Node* current = tail;
    std::cout << "[";
    while (current != nullptr) {
        std::cout << current->data;
        if (current->prev != nullptr) std::cout << ", ";
        current = current->prev;
    }
    std::cout << "]" << std::endl;
}

```

```

int size() const { return count; }

```

```

void clear() {
    while (head != nullptr) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    tail = nullptr;
    count = 0;
}
};

```

```

int main() {
    setlocale(LC_ALL, "ru");

```

```

    DoublyLinkedList<int> list;

```

```

    for (int i = 1; i <= 5; i++) {
        list.push_back(i * 10);
    }

```

```

    std::cout << "=== Двусвязный список ===" << std::endl;

```

```

std::cout << "Прямой порядок: ";
list::print();
std::cout << "Обратный порядок: ";
list::printReverse();

list::pop_back();
std::cout << "После pop_back: ";
list::print();

return 0;
}

```

Ожидаемый вывод:

=== Двусвязный список ===

Прямой порядок: [10, 20, 30, 40, 50]

Обратный порядок: [50, 40, 30, 20, 10]

После pop_back: [10, 20, 30, 40]

Пример 3. Итераторы и реверс списка (продвинутый уровень)

Задача: Реализовать итераторы для двусвязного списка и метод реверса.

```
#include <iostream>
```

```
#include <stdexcept>
```

```
template <typename T>
```

```
class DoublyLinkedList {
```

```
private:
```

```
    struct Node {
```

```
        T data;
```

```
        Node* prev;
```

```
        Node* next;
```

```
        Node(const T& value) : data(value), prev(nullptr), next(nullptr) {}
```

```
    };
```

```
    Node* head;
```

```
    Node* tail;
```

```
    int count;
```

```
public:
```

```
    // Итератор
```

```
    class Iterator {
```

```
    private:
```

```
        Node* ptr;
```

```
    public:
```

```

Iterator(Node* p = nullptr) : ptr(p) {}

T& operator*() { return ptr->data; }
Iterator& operator++() { ptr = ptr->next; return *this; }
Iterator operator++(int) { Iterator temp = *this; ptr = ptr->next; return
temp; }

Iterator& operator--() { ptr = ptr->prev; return *this; }
Iterator operator--(int) { Iterator temp = *this; ptr = ptr->prev; return
temp; }

bool operator==(const Iterator& other) const { return ptr == other.ptr; }
bool operator!=(const Iterator& other) const { return ptr != other.ptr; }
};

```

```

DoublyLinkedList() : head(nullptr), tail(nullptr), count(0) {}

```

```

~DoublyLinkedList() { clear(); }

```

```

void push_back(const T& value) {
    Node* newNode = new Node(value);
    if (tail == nullptr) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
    count++;
}

```

```

// Реверс списка (меняем местами указатели next и prev)

```

```

void reverse() {
    if (head == nullptr) return;

    Node* current = head;
    Node* temp = nullptr;

    while (current != nullptr) {
        // Меняем местами prev и next
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
    }
}

```

```

prev)        current = current->prev; // переходим к следующему (бывшему
            }

            // Меняем местами head и tail
            temp = head;
            head = tail;
            tail = temp;
        }

        Iterator begin() { return Iterator(head); }
        Iterator end() { return Iterator(nullptr); }

        void print() const {
            Node* current = head;
            std::cout << "[";
            while (current != nullptr) {
                std::cout << current->data;
                if (current->next != nullptr) std::cout << ", ";
                current = current->next;
            }
            std::cout << "]" << std::endl;
        }

        void clear() {
            while (head != nullptr) {
                Node* temp = head;
                head = head->next;
                delete temp;
            }
            tail = nullptr;
            count = 0;
        }

        int size() const { return count; }
    };

    int main() {
        setlocale(LC_ALL, "ru");

        DoublyLinkedList<int> list;
    }

```

```

for (int i = 1; i <= 5; i++) {
    list.push_back(i);
}

std::cout << "=== Итераторы и реверс ===" << std::endl;
std::cout << "Исходный список: ";
list.print();

std::cout << "Обход с помощью итератора: ";
for (auto it = list.begin(); it != list.end(); ++it) {
    std::cout << *it << " ";
}
std::cout << std::endl;

list.reverse();
std::cout << "После реверса: ";
list.print();

return 0;
}

```

Ожидаемый вывод:

=== Итераторы и реверс ===

Исходный список: [1, 2, 3, 4, 5]

Обход с помощью итератора: 1 2 3 4 5

После реверса: [5, 4, 3, 2, 1]

Индивидуальные задания

Базовый уровень (15 вариантов)

Общее задание: Реализовать класс односвязного списка с указанными методами.

№ вар.	Обязательные методы	Дополнительное задание
1	push_front, pop_front, print	Найти сумму всех элементов
2	push_back, pop_back, print	Найти количество элементов
3	push_front, push_back, print	Найти максимальный элемент
4	push_front, pop_front, find	Найти минимальный элемент

№ вар.	Обязательные методы	Дополнительное задание
5	push_back, pop_back, find	Проверить, есть ли заданное значение
6	push_front, push_back, pop_front	Вывести элементы через один
7	push_front, push_back, pop_back	Вывести элементы в обратном порядке (без реверса)
8	push_front, find, remove	Удалить все чётные элементы
9	push_back, find, remove	Удалить все нечётные элементы
10	push_front, push_back, clear	После очистки вывести размер
11	push_front, pop_front, size	Проверить, является ли список палиндромом
12	push_back, pop_back, empty	Скопировать список в другой
13	push_front, find, insert_after	Вставить элемент после заданного
14	push_back, find, insert_before	Вставить элемент перед заданным
15	push_front, push_back, getAt	Получить элемент по индексу (реализовать)

Средний уровень (15 вариантов)

Общее задание: Реализовать класс двусвязного списка с указанными методами и дополнительной функциональностью.

№ вар.	Дополнительная функциональность
1	Метод reverse() - разворот списка
2	Метод merge(DoublyLinkedList& other) - слияние двух отсортированных списков

№ вар.	Дополнительная функциональность
3	Метод <code>removeDuplicates()</code> - удаление дубликатов
4	Метод <code>getMiddle()</code> - получение среднего элемента (без вычисления размера)
5	Метод <code>hasCycle()</code> - проверка на наличие цикла
6	Метод <code>rotateLeft(int k)</code> - циклический сдвиг влево на k позиций
7	Метод <code>rotateRight(int k)</code> - циклический сдвиг вправо на k позиций
8	Метод <code>isPalindrome()</code> - проверка, является ли список палиндромом
9	Метод <code>intersection(DoublyLinkedList& other)</code> - пересечение двух списков
10	Метод <code>removeEverySecond()</code> - удаление каждого второго элемента
11	Метод <code>swapPairs()</code> - обмена соседних элементов
12	Метод <code>split()</code> - разделение списка на два: чётные и нечётные позиции
13	Метод <code>sort()</code> - сортировка списка (пузырьком или вставками)
14	Метод <code>toVector()</code> - преобразование в <code>std::vector</code>
15	Метод <code>fromVector(const std::vector<T>&)</code> - создание списка из вектора

Продвинутый уровень (15 вариантов)

Общее задание: Реализовать оба класса (`SinglyLinkedList` и `DoublyLinkedList`) с полной функциональностью и дополнительными требованиями.

№ вар.	Дополнительные требования
1	Реализовать итераторы для обоих списков (begin, end)
2	Реализовать операторы сравнения (==, !=) для списков
3	Реализовать оператор [] для доступа по индексу (с проверкой сложности)
4	Реализовать метод sort() (быстрая сортировка для двусвязного списка)
5	Реализовать метод mergeSort() для односвязного списка
6	Реализовать сериализацию: saveToFile и loadFromFile
7	Реализовать исключения для всех операций (выход за границы, удаление из пустого)
8	Реализовать std::initializer_list для удобной инициализации
9	Реализовать const_iterator для обоих списков
10	Реализовать reverse_iterator для двусвязного списка
11	Сравнить производительность операций односвязного и двусвязного списков
12	Реализовать кольцевой (циклический) список на основе двусвязного
13	Реализовать список с пропусками (skip list) на основе односвязного
14	Реализовать аллокатор для узлов (pool allocator)
15	Реализовать ленивое удаление (маркировка удалённых элементов)

Контрольные вопросы

1. Чем связный список отличается от динамического массива?
2. Какие операции в односвязном списке выполняются за $O(1)$?

3. Почему удаление из конца односвязного списка требует $O(n)$ времени?
4. Какие преимущества даёт хранение указателя tail в односвязном списке?
5. Чем двусвязный список отличается от односвязного?
6. Какие операции в двусвязном списке выполняются быстрее, чем в односвязном?
7. Что такое «голова» (head) и «хвост» (tail) списка?
8. Как удалить узел из середины односвязного списка?
9. Как удалить узел из середины двусвязного списка? Почему это проще?
10. Что произойдёт, если забыть обновить tail при удалении?
11. Как реализовать реверс односвязного списка?
12. Как обнаружить цикл в связном списке?
13. В каких случаях стоит использовать список вместо массива?
14. Какая дополнительная память требуется для каждого узла списка?
15. Почему доступ по индексу в списке имеет сложность $O(n)$?

Требования к отчёту

Титульный лист с названием работы, ФИО студента, номером группы, вариантом

Цель работы (формулируется самостоятельно)

Теоретическая часть (основные понятия: связный список, узел, head, tail)

Постановка задачи (текст задания согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода программы)

Анализ сложности (для каждой операции указать сложность)

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Базовый уровень (максимум 60 баллов)

Критерий	Баллы
Класс односвязного списка реализован корректно	20
Реализованы указанные методы	20

Критерий	Баллы
Корректное управление памятью (деструктор)	10
Демонстрационная программа показывает работу	5
Код отформатирован, есть комментарии	5

Средний уровень (максимум 80 баллов)

Критерий	Баллы
Все требования базового уровня	40
Класс двусвязного списка реализован корректно	15
Реализована дополнительная функциональность	15
Корректная работа с tail (для двусвязного)	10

Продвинутый уровень (максимум 100 баллов)

Критерий	Баллы
Все требования среднего уровня	60
Реализованы оба списка с полной функциональностью	15
Реализованы итераторы и операторы	15
Проведено сравнение производительности	5
Наличие ссылки на Git-репозиторий	5

Штрафные баллы

Нарушение	Штраф
Утечка памяти	-20
Неверная работа с указателями	-15

Нарушение	Штраф
Отсутствие обработки пустого списка	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60